Practical 1 - Demonstration of creating database.

```
CREATE TABLE student (
  id NUMBER PRIMARY KEY,
  name VARCHAR2(50),
  age NUMBER
);
```

Practical 2 - Create table insert 10 records in it.

```
-- Create Table
CREATE TABLE student (
  id NUMBER PRIMARY KEY,
  name VARCHAR2(50),
  age NUMBER
);

-- Insert Records
INSERT INTO student VALUES (1, 'Rahul', 20);
INSERT INTO student VALUES (2, 'Amit', 21);
INSERT INTO student VALUES (3, 'Neha', 19);
INSERT INTO student VALUES (4, 'Priya', 22);
INSERT INTO student VALUES (5, 'Rohan', 20);
INSERT INTO student VALUES (6, 'Sneha', 21);
INSERT INTO student VALUES (7, 'Karan', 23);
INSERT INTO student VALUES (8, 'Pooja', 20);
INSERT INTO student VALUES (9, 'Vikas', 22);
INSERT INTO student VALUES (10, 'Anjali', 19);

-- Save Changes
COMMIT;

-- View Data
SELECT * FROM student;
```

ID	NAME	AGE
1	Rahul	20
2	Amit	21
3	Neha	19
4	Priya	22
5	Rohan	20
6	Sneha	21
7	Karan	23
8	Pooja	20
9	Vikas	22
10	Anjali	19

Practical 3 - Demonstrate to INSERT, UPDATE, and DELETE, Records in Table.

```
-- Insert records
INSERT INTO student VALUES (11, 'Arjun', 21);
INSERT INTO student VALUES (12, 'Meena', 20);

-- Update record
UPDATE student
SET age = 22
WHERE id = 1;

-- Delete record
DELETE FROM student
WHERE id = 2;

-- Save changes
COMMIT;

-- Display table
SELECT * FROM student;
```

ID	NAME	AGE
1	Rahul	22
3	Neha	19
4	Priya	22
5	Rohan	20
6	Sneha	21
7	Karan	23
8	Pooja	20
9	Vikas	22
10	Anjali	19
11	Arjun	21
12	Meena	20

Practical 4 - Demonstrate to Alter Table (Add Column, Delete Column, Rename, Modify Column).

-- Add column

```
ALTER TABLE student
ADD email VARCHAR2(50);
```

-- Modify column

```
ALTER TABLE student
MODIFY name VARCHAR2(100);
```

-- Rename column

```
ALTER TABLE student
RENAME COLUMN email TO student_email;
```

-- Delete column

```
ALTER TABLE student
DROP COLUMN student_email;
```

-- Display table structure

```
DESC student;
```

```
Name                                      Null?    Type
----------------------------------------- -------- ----------------------------
ID                                        NOT NULL NUMBER
NAME                                               VARCHAR2(100)
AGE                                                NUMBER
```

Practical 5 - Demonstrate to SELECT with clauses (Simple and Parameterize).

```
-- Simple SELECT
SELECT * FROM student;
SELECT name, age FROM student;

-- WHERE clause
SELECT * FROM student
WHERE age > 20;

-- ORDER BY clause
SELECT * FROM student
ORDER BY age ASC;

-- GROUP BY clause
SELECT age, COUNT(*)
FROM student
GROUP BY age;

-- WHERE with AND condition
SELECT * FROM student
WHERE age > 20 AND name LIKE 'A%';
```

1. SELECT name, age FROM student;

NAME	AGE
Rahul	22
Neha	19
Priya	22
Rohan	20
Sneha	21
Karan	23
Pooja	20
Vikas	22
Anjali	19
Arjun	21
Meena	20

2. WHERE age > 20

ID	NAME	AGE
1	Rahul	22
4	Priya	22
6	Sneha	21
7	Karan	23
9	Vikas	22
11	Arjun	21

3. ORDER BY age ASC

ID	NAME	AGE
3	Neha	19
10	Anjali	19
5	Rohan	20
8	Pooja	20
12	Meena	20
6	Sneha	21
11	Arjun	21
1	Rahul	22
4	Priya	22
9	Vikas	22
7	Karan	23

4. GROUP BY age

AGE	COUNT(*)
19	2
20	3
21	2
22	3
23	1

5. WHERE age > 20 AND name LIKE 'A%'

ID	NAME	AGE
11	Arjun	21

Practical 6 - Demonstrate: i. WHERE Clause ii. GROUP BY clause iii. HAVING Clause

```
-- WHERE Clause
SELECT * FROM student
WHERE age > 20;

-- GROUP BY Clause
SELECT age, COUNT(*) AS total_students
FROM student
GROUP BY age;

-- HAVING Clause
SELECT age, COUNT(*) AS total_students
FROM student
GROUP BY age
HAVING COUNT(*) > 1;
```

1. WHERE age > 20

ID	NAME	AGE
1	Rahul	22
4	Priya	22
6	Sneha	21
7	Karan	23
9	Vikas	22
11	Arjun	21

2. GROUP BY age

AGE	COUNT(*)
19	2
20	3
21	2
22	3
23	1

3. HAVING Clause (COUNT(*) > 1)

AGE	TOTAL_STUDENTS
19	2
20	3
21	2
22	3

Practical 7 - Demonstrate integrity constraints: PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, and DEFAULT

```
-- Create Department table
CREATE TABLE department (
  dept_id NUMBER PRIMARY KEY,
  dept_name VARCHAR2(50) NOT NULL
);

-- Create Student table with constraints
CREATE TABLE student (
  id NUMBER PRIMARY KEY,
  name VARCHAR2(50) NOT NULL,
  age NUMBER CHECK (age >= 18),
  city VARCHAR2(50) DEFAULT 'Shahada',
  dept_id NUMBER,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

-- Insert records in department
INSERT INTO department VALUES (1, 'Computer');
INSERT INTO department VALUES (2, 'IT');

-- Insert records in student
INSERT INTO student (id, name, age, dept_id) VALUES (1, 'Rahul', 20, 1);
INSERT INTO student (id, name, age, dept_id) VALUES (2, 'Neha', 19, 2);
INSERT INTO student (id, name, age, dept_id) VALUES (3, 'Amit', 22, 1);

-- Save changes
COMMIT;
```

-- Display data

SELECT * FROM student;

SELECT * FROM department;

1. Department Table Output

DEPT_ID	DEPT_NAME
1	Computer
2	IT

2. Student Table Output

ID	NAME	AGE	CITY	DEPT_ID
1	Rahul	20	Shahada	1
2	Neha	19	Shahada	2
3	Amit	22	Shahada	1

Practical 8 - Demonstrate use of operators.

```
-- Arithmetic Operator
SELECT id, name, age, age + 2 AS future_age
FROM student;

-- Comparison Operator
SELECT * FROM student
WHERE age >= 21;

-- Logical Operators
SELECT * FROM student
WHERE age > 20 AND name LIKE 'A%';

SELECT * FROM student
WHERE age < 20 OR name LIKE 'P%';

-- NOT Operator
SELECT * FROM student
WHERE NOT age = 20;

-- BETWEEN Operator
SELECT * FROM student
WHERE age BETWEEN 20 AND 22;

-- IN Operator
SELECT * FROM student
WHERE name IN ('Rahul', 'Neha');

-- LIKE Operator
SELECT * FROM student
WHERE name LIKE 'R%';
```

Student Table Data (Reference)

ID	NAME	AGE	CITY	DEPT_ID
1	Rahul	20	Shahada	1
2	Neha	19	Shahada	2
3	Amit	22	Shahada	1

1. Arithmetic Operator (age + 2)

ID	NAME	AGE	FUTURE_AGE
1	Rahul	20	22
2	Neha	19	21
3	Amit	22	24

2. Comparison Operator (age >= 21)

ID	NAME	AGE	CITY	DEPT_ID
3	Amit	22	Shahada	1

3. Logical Operator (AND)

ID	NAME	AGE	CITY	DEPT_ID
3	Amit	22	Shahada	1

4. Logical Operator (OR)

ID	NAME	AGE	CITY	DEPT_ID
2	Neha	19	Shahada	2

5. NOT Operator

ID	NAME	AGE	CITY	DEPT_ID
2	Neha	19	Shahada	2
3	Amit	22	Shahada	1

6. BETWEEN Operator

ID	NAME	AGE	CITY	DEPT_ID
1	Rahul	20	Shahada	1
3	Amit	22	Shahada	1

7. IN Operator

ID	NAME	AGE	CITY	DEPT_ID
1	Rahul	20	Shahada	1
2	Neha	19	Shahada	2

8. LIKE Operator

ID	NAME	AGE	CITY	DEPT_ID
1	Rahul	20	Shahada	1

Practical 9 - Demonstrate joins (Inner joins, Equi, Non Equi, Self-join & Outer Joins).

```
-- Create tables
CREATE TABLE department (
  dept_id NUMBER PRIMARY KEY,
  dept_name VARCHAR2(50)
);

CREATE TABLE student (
  id NUMBER PRIMARY KEY,
  name VARCHAR2(50),
  age NUMBER,
  dept_id NUMBER
);

-- Insert data (Oracle style)
INSERT INTO department VALUES (1, 'Computer');
INSERT INTO department VALUES (2, 'IT');
INSERT INTO department VALUES (3, 'Mechanical');

INSERT INTO student VALUES (1, 'Rahul', 20, 1);
INSERT INTO student VALUES (2, 'Amit', 21, 2);
INSERT INTO student VALUES (3, 'Neha', 22, 1);
INSERT INTO student VALUES (4, 'Pooja', 20, NULL);

COMMIT;

-- INNER JOIN
SELECT s.name, d.dept_name
FROM student s
INNER JOIN department d
```

```
ON s.dept_id = d.dept_id;

-- EQUI JOIN
SELECT s.name, d.dept_name
FROM student s, department d
WHERE s.dept_id = d.dept_id;

-- SELF JOIN
SELECT A.name AS Student1, B.name AS Student2
FROM student A, student B
WHERE A.dept_id = B.dept_id AND A.id <> B.id;

-- NON-EQUI JOIN
SELECT s.name, s.age, d.dept_name
FROM student s, department d
WHERE s.age > d.dept_id;

-- LEFT OUTER JOIN
SELECT s.name, d.dept_name
FROM student s
LEFT OUTER JOIN department d
ON s.dept_id = d.dept_id;

-- RIGHT OUTER JOIN
SELECT s.name, d.dept_name
FROM student s
RIGHT OUTER JOIN department d
ON s.dept_id = d.dept_id;
```

Table Data (Reference)

Student

ID	NAME	AGE	DEPT_ID
1	Rahul	20	1
2	Amit	21	2
3	Neha	22	1
4	Pooja	20	NULL

Department

DEPT_ID	DEPT_NAME
1	Computer
2	IT
3	Mechanical

1. INNER JOIN, EQUI JOIN

NAME	DEPT_NAME
Rahul	Computer
Amit	IT
Neha	Computer

2. SELF JOIN

STUDENT1	STUDENT2
Rahul	Neha
Neha	Rahul

3. NON-EQUI JOIN (s.age > d.dept_id)

NAME	AGE	DEPT_NAME
Rahul	20	Computer
Rahul	20	IT
Rahul	20	Mechanical
Amit	21	Computer
Amit	21	IT
Amit	21	Mechanical
Neha	22	Computer
Neha	22	IT
Neha	22	Mechanical
Pooja	20	Computer
Pooja	20	IT
Pooja	20	Mechanical

5. LEFT OUTER JOIN

NAME	DEPT_NAME
Rahul	Computer
Amit	IT
Neha	Computer
Pooja	NULL

6. RIGHT OUTER JOIN

NAME	DEPT_NAME
Rahul	Computer
Neha	Computer
Amit	IT
NULL	Mechanical

Practical 10 - Demonstrate the use of Sub-Queries.

```
-- Example 1: Single-row Sub-query
SELECT name, age
FROM student
WHERE age = (SELECT MAX(age) FROM student);

-- Example 2: Multiple-row Sub-query
SELECT name
FROM student
WHERE dept_id IN (SELECT dept_id FROM department);

-- Example 3: Sub-query with comparison
SELECT name, age
FROM student
WHERE age > (SELECT AVG(age) FROM student);


-- Example 4: Sub-query in SELECT clause
SELECT name, age,
      (SELECT AVG(age) FROM student) AS average_age
FROM student;
```

Student Table (Reference)

ID	NAME	AGE	DEPT_ID
1	Rahul	20	1
2	Amit	21	2
3	Neha	22	1
4	Pooja	20	NULL

1. Single-row Sub-query (MAX age)

NAME	AGE
Neha	22

2. Multiple-row Sub-query (IN)

NAME
Rahul
Amit
Neha

3. Sub-query with Comparison (AVG age)

NAME	AGE
Amit	21
Neha	22

4. Sub-query in SELECT clause

NAME	AGE	AVERAGE_AGE
Rahul	20	20.75
Amit	21	20.75
Neha	22	20.75
Pooja	20	20.75

Practical 11 - Write down SQL by using i. Aggregate functions ii. Date functions iii. String functions.

-- Aggregate Functions

```
SELECT COUNT(*) AS total_students FROM student;

SELECT AVG(age) AS average_age FROM student;
SELECT MAX(age) AS maximum_age FROM student;
SELECT MIN(age) AS minimum_age FROM student;
```

-- Date Functions

```
SELECT SYSDATE AS current_date FROM dual;

SELECT SYSDATE AS current_date_time FROM dual;
```

-- String Functions

```
SELECT UPPER(name) AS upper_name FROM student;
SELECT LOWER(name) AS lower_name FROM student;
SELECT LENGTH(name) AS name_length FROM student;

SELECT name || ' - Student' AS full_info FROM student;
```

Student Table (Reference)

ID	NAME	AGE	DEPT_ID
1	Rahul	20	1
2	Amit	21	2
3	Neha	22	1
4	Pooja	20	NULL

1. Aggregate Functions

- **COUNT(*)**

TOTAL_STUDENTS
4

- **AVG(age)**

AVERAGE_AGE
20.75

- **MAX(age)**

MAXIMUM_AGE
22

- **MIN(age)**

MINIMUM_AGE
20

2. Date Functions

- **SYSDATE (Current Date)**

CURRENT_DATE
28-MAR-2026

- **SYSDATE (Date & Time)**

CURRENT_DATE_TIME
28-MAR-2026 05:30:00 PM

3. String Functions

- **UPPER(name)**

UPPER_NAME
RAHUL
AMIT
NEHA
POOJA

- **LOWER(name)**

LOWER_NAME
rahul
amit
neha
pooja

- **LENGTH(name)**

NAME_LENGTH
5
4
4
5

- **Concatenation (||)**

FULL_INFO
Rahul - Student
Amit - Student
Neha - Student
Pooja - Student